AGM 1기 · WORKBOOK

웹 기본지식 워크북

HTML 구조부터 웹과 통신의 이해까지
— 슬라이드 요약본을 "읽고 이해하는 자료"로 풀어쓴 상세 해설서

구성 · 4개 파트 + 용어사전 + 연습문제 + 실습 + 정답 해설
사용법 · 순서대로 읽고, 각 파트 끝의 **확인 문제**를 직접 풀어보세요. 막히면 맨 뒤 정답 해설로 가면 됩니다.

이 워크북을 읽는 법

원본 슬라이드는 발표용이라 문장이 짧고, 설명이 빠져 있는 부분이 많습니다. 이 워크북은 그 빈칸을 채우기 위해 만들었습니다. 슬라이드의 순서와 용어는 그대로 따르되, 각 개념마다 아래 순서로 풀어 썼습니다.

표시	의미
핵심 정의	외워야 할 한두 줄 정의
쉽게 말하면	일상에 빗댄 비유. 이해가 안 될 때 이 부분부터 읽으세요
자세히 보기	슬라이드에 없던 보충 설명과 실제 예시
주의	초보자가 자주 틀리는 부분
직접 해보기	브라우저에서 바로 확인할 수 있는 실습

목차

PART 0 · 한 장으로 보는 전체 그림 — 이야기로 먼저 이해하기

PART 1 · 웹은 어떻게 작동하나 — 인터넷·웹, 클라이언트·서버, URL, DNS, HTTP

PART 2 · HTML 문서 구조 — 태그·요소·속성, 시맨틱, DOM

PART 3 · HTML · CSS · JS 역할 분담 — 구조·표현·동작

PART 4 · 브라우저 렌더링과 개발자도구 — 파싱 → 렌더 트리 → 레이아웃 → 페인트

부록 A · 핵심 용어 사전

부록 B · 흔한 오해 바로잡기

부록 C · 종합 연습문제

부록 D · 미니 프로젝트 (내 첫 웹 페이지)

부록 E · 정답 및 해설

추천 학습 방법

① 한 파트를 읽는다 → ② 책을 덮고 "내 말로" 한 문장씩 설명해 본다 → ③ 확인 문제를 푼다 → ④ 크롬에서 F12를 눌러 실제로 확인한다. 이 네 단계를 지키면 이해가 훨씬 오래 갑니다.

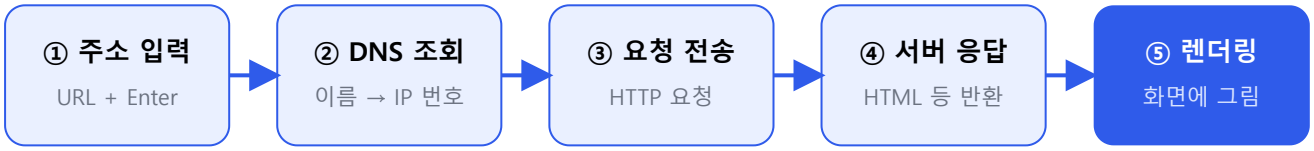
PART 0

한 장으로 보는 전체 그림

세부 내용에 들어가기 전에, 이 강의 전체를 이야기 하나로 먼저 잡아둡니다.

"주소창에 주소를 치고 Enter를 누르면" 무슨 일이 생길까?

여러분이 `https://shop.example.com/products/shoes` 를 주소창에 입력하고 Enter를 누른다고 합시다. 우리 눈에는 0.5초 만에 쇼핑몰 화면이 뜨는 것이 전부지만, 그 짧은 순간 안에서는 사실 아래와 같은 일들이 차례로 벌어집니다.



이 다섯 단계가 이 강의의 뼈대입니다

그림 0-1. Enter 이후의 다섯 단계

쉽게 말하면 — 식당 주문 이야기

당신(**클라이언트**)이 식당에서 메뉴판의 이름을 보고(**URL**), 그 이름에 해당하는 주방의 위치를 안내판에서 찾은 다음(**DNS**), 점원에게 주문서를 건네고(**HTTP 요청**), 주방(**서버**)이 요리를 내오면(**응답**), 접시 위에 음식을 예쁘게 차려 먹을 준비를 합니다(**렌더링**).

이 강의에서 배우는 것 한 줄 정리

파트	한 줄 요약
1. 웹은 어떻게 작동하나	브라우저(클라이언트)와 서버가 HTTP라는 약속으로 대화하고, 주소는 URL, 서버 찾기는 DNS가 맡는다.
2. HTML 문서 구조	서버가 보내주는 HTML은 태그로 이루어진 '뼈대'이며, 태그가 겹치며 나무 구조를 만든다.
3. HTML·CSS·JS	구조는 HTML, 꾸밈은 CSS, 동작은 JavaScript. 셋은 역할이 다르다.
4. 브라우저 렌더링	브라우저가 코드를 해석해 DOM을 만들고 화면에 그린다. 개발자도구(F12)로 이 모든 걸 볼 수 있다.

PART 1

웹은 어떻게 작동하나

주소창에 입력하고 Enter를 누르는 순간, 화면 뒤에서 일어나는 일을 단계별로 따라갑니다.

1-1. '인터넷'과 '웹'은 같은 말이 아니다

핵심 정의

인터넷(Internet): 전 세계 컴퓨터를 케이블·무선으로 연결한 거대한 통신망(기반 시설).
웹(World Wide Web): 그 인터넷 위에서 브라우저로 문서·페이지를 주고받는 *서비스 중 하나*.

두 단어를 혼용해서 쓰는 사람이 많지만 "층"이 다릅니다. 인터넷이 바닥에 깔린 도로라면, 웹은 그 도로 위를 달리는 여러 서비스 중 하나입니다. 같은 도로를 이메일, 카카오톡 같은 메신저, 온라인 게임, 스마트폰 앱도 함께 사용합니다.

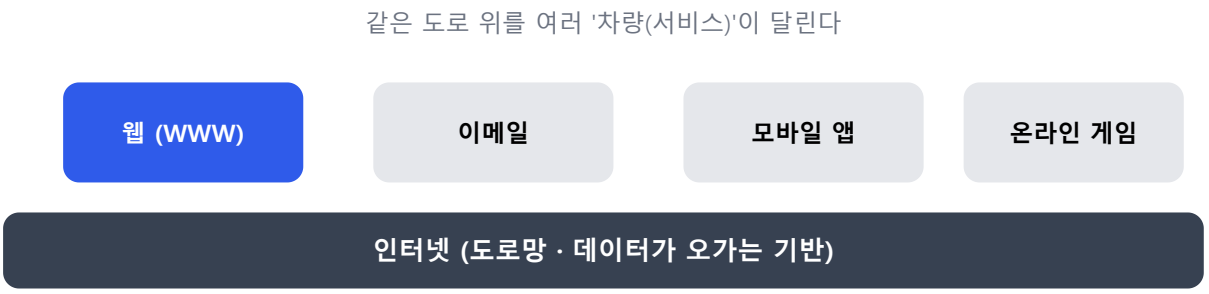


그림 1-1. 인터넷(도로)과 웹(그 위의 서비스)

쉽게 말하면

인터넷은 **도로**, 웹은 그 위를 달리는 **택배 트럭**. 같은 도로로 버스(이메일)와 오토바이(앱)도 다닙니다.

자세히 보기

웹은 1989~1990년경 팀 버너스리가 "문서끼리 링크로 연결하자"는 아이디어로 만들었습니다. 인터넷 자체는 그보다 훨씬 오래전부터 있었습니다. 즉 "인터넷이 먼저, 웹은 그 위에 나중에 얹힌 서비스"라는 순서도 둘이 다르다는 증거입니다.

1-2. 클라이언트와 서버: 묻는 쪽, 답하는 쪽

핵심 정의

클라이언트: 요청(request)을 보내는 쪽. 가장 대표적인 것이 **브라우저**(크롬, 엣지, 사파리 등).
서버: 요청을 받아 필요한 파일·데이터를 찾아 응답(response)으로 돌려주는 쪽. 항상 켜져서 대기한다.

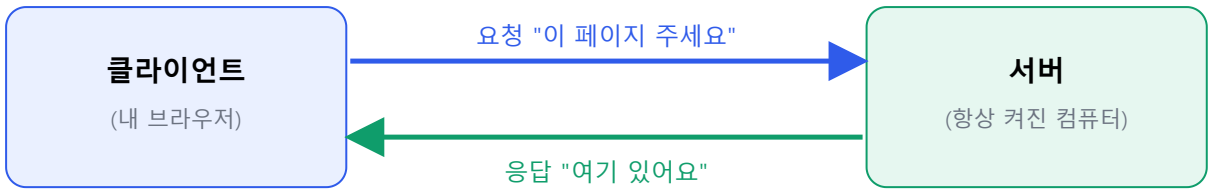


그림 1-2. 요청과 응답

쉽게 말하면

식당과 같습니다. **손님(클라이언트)**이 주문하면 **주방(서버)**이 요리를 만들어 내옵니다. 손님이 없어도 주방은 문을 열고 대기합니다.

주의 — "서버는 특별한 기계"가 아니다

서버는 신비한 장비가 아니라 "요청에 응답하도록 프로그램을 켜 둔 컴퓨터"일 뿐입니다. 이름도 역할(role)을 가리키는 말입니다. 같은 컴퓨터가 어떤 때는 서버, 어떤 때는 클라이언트가 될 수도 있습니다. 또한 "웹 서버"는 컴퓨터 자체를 뜻하기도 하고, 그 안에서 실행되는 프로그램을 뜻하기도 합니다.

1-3. 한눈에 보는 전체 흐름 (5단계)

PART 0에서 본 다섯 단계를 한 번 더 정리합니다. 이후 1-4부터 1-9까지는 이 단계를 하나씩 풀어 설명합니다.

단계	이름	무슨 일	관련 개념
①	주소 입력	URL을 입력하고 Enter	URL
②	DNS 조회	도메인 이름을 IP 주소로 변환	DNS
③	요청 전송	서버에 HTTP 요청을 보냄	HTTP, 메서드
④	서버 응답	서버가 HTML 등 파일을 돌려줌	상태 코드
⑤	렌더링	받은 코드를 해석해 화면에 그림	DOM, CSS

1-4. 단계 ① 주소 해부: URL을 뜯어보면

핵심 정의

URL(Uniform Resource Locator)은 인터넷에 있는 어떤 자원(resource)의 위치를 나타내는 주소입니다. 집 주소처럼 일정한 형식이 있습니다.



그림 1-3. URL의 네 부분

부분	예시	의미
프로토콜	`https://`	"어떤 약속(방식)으로 통신할지". 웹에서는 거의 `http` 또는 `https`.
도메인	`shop.example.com`	"어느 서버에 접속할지". 사람이 읽을 수 있는 서버 이름. `shop`은 서브도메인, `example.com`은 사이트의 본 이름입니다.
경로	`/products/shoes`	"그 서버 안에서 어떤 자원을 원하는지". 폴더 경로처럼 `/`로 구분합니다.
쿼리	`?color=red`	"추가 조건". `?`로 시작하고 `이름=값` 형태. 여러 개는 `&`로 연결 (예: `?color=red&size=270`).

슬라이드에는 없지만 알아두면 좋은 부분

- **포트(port)**: 서버 안의 "출입구 번호". `https://example.com:8080/` 처럼 도메인 뒤에 `:번호`로 쓰며, 생략하면 http는 80번, https는 443번이 자동 적용됩니다.
- **프래그먼트(fragment)**: `#section2` 처럼 `#` 뒤에 붙으며, 페이지 안의 특정 위치로 이동합니다. 서버로는 전송되지 않고 브라우저 안에서만 쓰입니다.

> **쉽게 말하면**
> 택배 주소와 똑같습니다. "(프로토콜) 일반 택배로 → (도메인) 서울시 ○○아파트 → (경로) 101동 502호 → (쿼리) 부재 시 경비실에 맡겨주세요".

> **직접 해보기 ①**
> 구글에서 아무 단어나 검색한 뒤 주소창을 보세요. `https://www.google.com/search?q=검색어` 처럼 `?q=검색어`가 쿼리로 붙어 있는 것을 볼 수 있습니다. 주소창에서 검색어 부분만 바꿔서 Enter를 누르면 다른 검색 결과가 나옵니다. 쿼리가 "서버에 전달하는 추가 조건"이라는 의미를 직접 느낄 수 있습니다.

1-5. 단계 ② 주소 찾기: DNS는 인터넷의 전화번호부

핵심 정의

DNS(Domain Name System)는 사람이 외우기 쉬운 **도메인 이름**(`example.com`)을 컴퓨터가 쓰는 **IP 주소**(`93.184.216.34`)로 바꿔주는 시스템입니다.

컴퓨터와 서버는 네트워크에서 서로를 숫자(IP 주소)로 식별합니다. 하지만 사람은 숫자를 외우기 어렵기 때문에 이름을 쓰고, 그 이름을 숫자로 번역해 주는 안내 서비스가 필요합니다. 그게 DNS입니다.

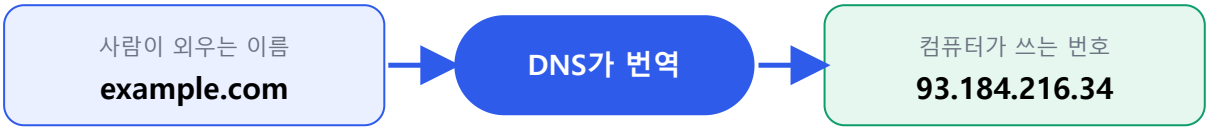


그림 1-4. DNS의 역할

쉽게 말하면

"홍길동"이라는 이름만 아는 상태에서 전화를 걸려면 전화번호부에서 번호를 찾아야 합니다.
이름만으로는 전화를 걸 수 없고, **번호(IP)**가 있어야 연결됩니다. DNS는 바로 그 전화번호부입니다.

자세히 보기 — DNS 조회는 실제로 이렇게 진행된다

"전화번호부 하나"로 설명했지만, 실제로는 가까운 곳부터 차례로 물어보는 구조입니다. 매번 먼 곳까지 묻지 않도록 **캐시**(기억해 두기)를 적극 활용합니다.

1. **브라우저 캐시**: 최근에 찾아본 주소가 있는지 확인
2. **운영체제(OS) 캐시**: 내 컴퓨터가 기억하는 주소 확인
3. **리졸버(Resolver)**: 통신사나 공용 DNS(예: 8.8.8.8)에 질문
4. 모르면 **루트 → TLD(.com 등) → 해당 도메인의 권한 서버** 순으로 물어서 최종 IP를 얻음
5. 결과를 돌려받아 캐시에 저장 (다음엔 훨씬 빠름)

직접 해보기 ②

PowerShell(또는 명령 프롬프트)에서 `nslookup example.com` 을 입력해 보세요. 도메인에 대응하는 IP 주소가 출력됩니다. DNS가 실제로 하는 일을 눈으로 볼 수 있습니다.

1-6. 단계 ③ 대화 규칙: HTTP는 브라우저와 서버의 공통 언어

핵심 정의

HTTP(HyperText Transfer Protocol)는 요청과 응답을 어떤 형식과 순서로 주고받을지 정한 **약속(프로토콜)**입니다. 약속이 같기 때문에 어떤 브라우저든 어떤 서버든 서로 대화할 수 있습니다.

한국 사람과 영국 사람이 말이 안 통하는 것처럼, 규칙이 다르면 컴퓨터끼리도 대화가 안 됩니다. HTTP는 전 세계가 합의한 "공통 언어"라서 크롬으로 어느 서버에든 접속할 수 있습니다.

HTTP 요청과 응답은 실제로 어떻게 생겼을까?

겉으로는 안 보이지만, 사실은 다음과 같은 텍스트가 오갑니다.

▶ 요청 (브라우저 → 서버)

```
GET /products/shoes?color=red HTTP/1.1
Host: shop.example.com
User-Agent: Chrome/120
Accept: text/html
```

- 첫 줄: **메서드** + **경로** + 버전
- 아래: **헤더**(부가 정보)

◀ 응답 (서버 → 브라우저)

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256

<!DOCTYPE html>
<html> ... </html>
```

- 첫 줄: **상태 코드**
- 헤더 뒤 빈 줄 다음이 **본문**(HTML)

HTTP vs HTTPS

	HTTP	HTTPS
의미	요청·응답의 형식과 순서를 규정	**HTTP + 암호화** (S = Secure)
내용 보호	내용이 그대로 오가서 중간에서 **엿볼 수 있음**	내용을 암호화해 중간에서 **읽을 수 없음**
기본 포트	80	443
현재	점점 사라지는 중	사실상 필수

> **쉽게 말하면**
> HTTP는 **엽서**입니다. 우체부도 이웃도 내용을 읽을 수 있죠. HTTPS는 **봉인된 편지**입니다. 중간에서 가로채도 열어볼 수 없고, 받는 사람만 읽을 수 있습니다.

> **확인 팁**
> 주소창 왼쪽에 **자물쇠 아이콘**이 보이면 HTTPS로 연결된 것입니다. 로그인·결제 화면에서 자물쇠가 없다면 비밀번호를 입력하지 마세요.

> **주의**
> HTTP의 'Stateless(무상태)' 특성: HTTP는 기본적으로 이전 요청을 기억하지 못합니다. 그래서 "로그인 상태 유지"를 위해 쿠키·세션 같은 별도 장치를 씁니다. 지금은 "서버는 매번 새로 묻는 것으로 인식한다" 정도만 알아두면 충분합니다.

1-7. HTTP 메서드: 무엇을 하려는가

요청을 보낼 때는 "**무엇을 하고 싶은지**"를 함께 알려야 합니다. 그 종류가 **HTTP 메서드**입니다. 데이터 처리의 4가지 기본 동작인 **CRUD**(Create·Read·Update·Delete)에 대응됩니다.

메서드	동작	CRUD	쓰이는 상황
`GET`	조회	Read	페이지 열기, 검색 결과 보기, 이미지 불러오기
`POST`	전송·생성	Create	회원가입, 글쓰기, 로그인 정보 보내기
`PUT`	수정	Update	프로필 변경, 게시글 수정
`DELETE`	삭제	Delete	게시글 삭제, 계정 탈퇴

쉽게 말하면 — 도서관
GET = 책 **찾아보기**, POST = 새 책 **등록하기**, PUT = 책 정보 **고치기**, DELETE = 책 **폐기하기**.

자세히 보기

- 평소에 링크를 클릭하거나 주소를 입력해 접속하는 것은 모두 **GET** 요청입니다.
- **GET은 주소(쿼리)에 정보가 드러나고, POST는 본문(body)에 정보를 담아 보냅니다.** 그래서 비밀번호 같은 민감한 정보는 GET이 아니라 POST로 보냅니다.
- PUT은 "통째로 교체", 일부만 고칠 때는 `PATCH` 라는 메서드도 있습니다. 여기서는 존재만 알아두세요.

1-8. 단계 ④ 응답 읽기: 상태 코드는 서버가 알려주는 결과

서버는 응답의 맨 앞에 **3자리 숫자**로 "요청이 어떻게 처리되었는지" 알려줍니다. 첫 자리 숫자만 봐도 큰 의미를 알 수 있습니다.

범위	의미	대표 코드	설명
2xx	성공	`200 OK`	요청이 잘 처리되었다 (`201 Created` 는 새로 만들었다)
3xx	이동 (리다이렉트)	`301 Moved Permanently`	주소가 바뀌었으니 다른 곳으로 가라
4xx	요청이 잘못됨	`404 Not Found` , `403 Forbidden` , `401 Unauthorized`	없는 페이지, 권한 없음 등 **클라이언트 쪽 문제**
5xx	서버 오류	`500 Internal Server Error` , `503 Service Unavailable`	서버 내부에서 문제가 생김 (서버가 터짐)

구분법

4로 시작하면 내 잘못(주소 오타 등), **5로 시작하면 서버 잘못**. 웹 서핑 중 **404 Not Found**를 본 적이 있다면 이미 상태 코드를 경험한 것입니다.

쉽게 말하면

식당 직원의 대답과 같습니다. 2xx "네, 여기 있습니다." / 3xx "그 메뉴는 옆 매장으로 옮겼어요." / 4xx "그런 메뉴는 없는데요?"(주문 실수) / 5xx "죄송해요, 주방에 불이 났어요."(식당 사정)

1-9. PART 1 다시 연결해 보기: 한 번의 요청 따라가기

아래는 `https://shop.example.com/products/shoes?color=red` 를 입력했을 때의 전체 과정을 하나로 이은 것입니다.

1. **URL 해석** — 브라우저가 주소를 프로토콜(https) / 도메인(shop.example.com) / 경로(/products/shoes) / 쿼리(color=red)로 나눈다.
2. **DNS 조회** — `shop.example.com` 의 IP 주소를 찾는다. (예: 93.184.216.34)
3. **연결 및 요청** — 그 IP의 서버(포트 443)에 접속하고, HTTPS로 암호화된 채널을 만든 뒤 `GET /products/shoes?color=red` 요청을 보낸다.
4. **서버 처리 및 응답** — 서버가 빨간 신발 목록 페이지를 준비해 `200 OK` 와 함께 HTML을 돌려준다.
5. **렌더링** — 브라우저가 HTML을 읽고, 안에 적힌 CSS·JS·이미지도 추가로 요청해 받아 화면을 그린다. (PART 4에서 상세히)

포인트

페이지 하나를 여는 데도 요청은 한 번이 아닙니다. HTML을 받은 뒤, 그 안에 적힌 이미지·CSS·JS 파일 각각이 추가 요청으로 이어집니다. 이것을 Network 탭(4-2)에서 직접 볼 수 있습니다.

🖍️ PART 1 확인 문제

Q1. 인터넷과 웹의 관계를 "도로"와 "트럭" 비유 없이 자기 말로 한 문장으로 설명해 보세요.

Q2. 다음 URL을 프로토콜 / 도메인 / 경로 / 쿼리로 나누어 쓰세요.

`https://news.example.org/sports/soccer?page=2`

프로토콜
도메인
경로
쿼리

Q3. 다음 상황에 맞는 HTTP 메서드는? (a) 쇼핑몰 상품 목록 보기 _____ (b) 새 게시글 작성 _____ (c) 내 프로필 사진 변경 _____ (d) 댓글 삭제 _____

Q4. 어떤 사이트에서 **404**가 나왔을 때와 **500**이 나왔을 때, 각각 누구 쪽 문제일 가능성이 큰가요?

PART 2

HTML 문서 구조

서버가 보내주는 바로 그 HTML — 웹 페이지의 뼈대가 실제로 어떻게 생겼는지 뜯어봅니다.

2-1. HTML은 페이지의 '뼈대'를 만든다

핵심 정의

HTML(HyperText Markup Language)은 텍스트에 "이건 제목, 이건 문단, 이건 링크"라고 **표시(마크업)**해서 문서에 의미와 구조를 부여하는 언어입니다.

여기서 'Markup'은 형광펜이나 교정 표시처럼 글 위에 "표시를 단다"는 뜻입니다. 같은 글자라도 어떤 표시를 달았느냐에 따라 제목이 되기도 하고 링크가 되기도 합니다.

✓ 이런 것이다

내용에 **의미와 구조**를 부여하는 표시 언어.
무엇이 제목이고 목록이고 표인지 정한다.

✗ 이런 게 아니다

계산·조건 판단 같은 로직을 처리하는 **프로그래밍 언어가 아니다**. 움직임은 JS, 꾸밈은 CSS의 몫.

쉽게 말하면

집으로 치면 HTML은 **골조와 방 배치**입니다. 페인트와 가구(CSS), 전기·수도(JS)는 그 다음에 얹힙니다. 골조가 없으면 아무것도 올릴 수 없습니다.

2-2. 모든 HTML 문서의 공통 골격

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <title>페이지 제목</title>
  </head>
  <body>
    <h1>안녕하세요</h1>
  </body>
</html>
```

※ 슬라이드의 예시를 약간 확장했습니다(`lang`, `meta charset` 추가). 실제 웹 문서에서는 이 정도가 기본입니다.

줄	설명
`<!DOCTYPE html>`	"이 문서는 HTML5 문서입니다"라는 **선언**. 맨 첫 줄에 쓰며, 브라우저가 문서를 올바른 방식으로 해석하게 합니다.
`<html lang="ko">`	문서 전체를 감싸는 **뿌리(root) 요소**. `lang="ko"`는 "한국어 문서"라는 표시(검색엔진·화면낭독기에 도움).
`<head>`	**화면 밖 정보**. 제목, 글자 인코딩, 연결할 CSS 파일 등 "문서에 대한 정보"를 담는다. 본문에는 직접 보이지 않습니다.
`<meta charset="UTF-8">`	글자 인코딩 지정. 빠뜨리면 한글이 깨져 보일 수 있습니다.
`<title>`	**브라우저 탭에 표시되는 제목**이자 검색 결과의 제목으로 쓰입니다.
`<body>`	실제로 **화면에 보이는 내용**이 담기는 곳.

> **쉽게 말하면**
> **head**는 책의 표지·판권 정보, **body**는 책의 본문 페이지입니다.

2-3. 3대 구성 단위: 태그 · 요소 · 속성

HTML을 읽을 때 가장 먼저 구분해야 할 세 가지 용어입니다. 이 용어가 헷갈리면 이후 설명이 전부 어려워지므로 확실히 짚고 넘어갑니다.

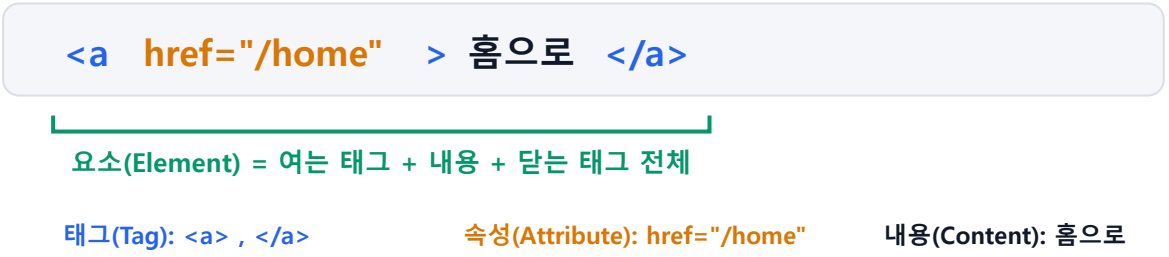


그림 2-1. 하나의 링크 요소 해부

용어	정의	쉬운 설명 / 주의점
태그	`<a>`, `</a>` 처럼 꺾쇠(< >)로 감싼 **표시**. 여는 태그와 닫는 태그가 한 쌍.	닫는 태그에는 이름 앞에 `/`가 붙는다. 여닫음을 맞춰야 "여기서부터 여기까지가 링크"라는 범위가 정해진다.
요소	여는 태그 + 내용 + 닫는 태그 **전체 한 덩어리**.	태그는 "표시", 요소는 "표시가 감싼 덩어리 전체". 엄밀히는 다르지만 일상에서는 혼용하기도 한다.
속성	요소에 붙는 **추가 정보**. `이름="값"` 형태로 여는 태그 안에 쓴다.	`href`(어디로 갈지), `src`(이미지 파일 경로), `alt`(이미지 설명), `class` / `id`(CSS·JS가 찾을 이름).

속성이 있는 예시들

```
<a href="https://example.com">예제 사이트</a>     <!-- href: 이동할 주소 -->
<img src="cat.jpg" alt="회색 고양이 사진">          <!-- src: 파일, alt: 대체 설명 -->
<p class="notice" id="first">공지입니다</p>        <!-- class·id: 이름표 -->
```

주의 — 닫는 태그가 없는 요소도 있다

`<img>`, `<br>`, `<meta>`, `<input>` 처럼 내용을 감쌀 필요가 없는 요소는 닫는 태그가 없습니다(빈 요소, void element). 모든 태그가 쌍이라고 외우지는 마세요.

자세히 보기 — 주석

`<!-- 이렇게 -->` 쓰면 화면에는 나타나지 않는 메모가 됩니다. 코드 설명이나 임시로 빼둘 때 씁니다.

2-4. 먼저 익힐 자주 쓰는 기본 태그

태그	이름	설명
`<h1>~<h6>`	제목(Heading)	숫자가 **작을수록 크고 중요**. h1은 페이지의 대제목(보통 한 페이지에 하나), h2는 큰 소제목, h3은 그 아래 소제목…
`<p>`	문단(Paragraph)	본문 텍스트 덩어리. 문단 앞뒤로 자동으로 간격이 생김
`<a>`	링크(Anchor)	다른 페이지로 이동. 목적지는 `href` 속성에
`<img>`	이미지(Image)	이미지를 삽입. `src`, `alt` 속성 사용
`<ul> <li>`	목록(Unordered List) / 항목(List Item)	`ul`은 점 목록 전체, `li`는 각 항목. 번호 목록은 `ol`
`<div>`	구획(Division)	아무 의미 없이 요소를 **묶는 상자**. 레이아웃 구획용

한 번에 써 보기

```
<h1>오늘의 추천 맛집</h1>
<p>이번 주에 다녀온 곳을 소개합니다.</p>

<ul>
  <li>김밥천국</li>
  <li>돈까스 가게</li>
</ul>

<img src="food.jpg" alt="맛집 음식 사진">
<p>더 보려면 <a href="https://example.com">여기</a>를 누르세요.</p>
```

주의 1

`<h1>` ~ `<h6>`를 "글자 크기 조절용"으로 쓰지 마세요. 크기가 아니라 **중요도(계층)**를 나타내는 태그입니다. 크기는 CSS가 정합니다.

주의 2

`<img>`의 `alt`는 시각장애인용 화면낭독기와 이미지 로딩 실패 시를 위한 설명입니다. 반드시 넣는 습관을 들이세요.

주의 3

HTML은 줄바꿈·연속 공백을 화면에 그대로 반영하지 않습니다. 소스에서 엔터를 여러 번 쳐도 화면에서는 한 칸 공백으로 합쳐집니다. 줄바꿈은 `<br>`, 문단 구분은 `<p>`를 사용하세요.

2-5. 시맨틱 태그: 이름에 뜻이 있다

핵심 정의

시맨틱(semantic) 태그는 이름만 보고도 "이 영역이 무슨 역할인지" 알 수 있는 태그입니다. 모양이 아니라 **의미**를 담습니다.

상자 역할을 하는 `<div>` 하나로도 페이지를 만들 수 있지만, 그러면 어디가 머리말이고 어디가 본문인지 사람이 아닌 프로그램(검색엔진, 화면낭독기, AI)은 알 수가 없습니다. 의미가 담긴 태그를 쓰면 이들이 구조를 이해합니다.

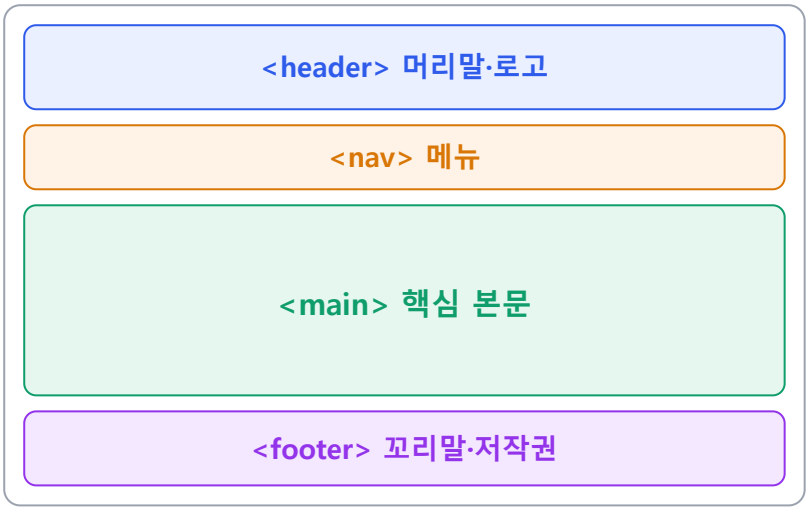


그림 2-2. 시맨틱 태그로 나눈 일반적인 페이지 구조

태그	의미
`<header>`	머리말, 로고, 상단 영역
`<nav>`	메뉴·내비게이션 링크 모음
`<main>`	페이지의 핵심 본문 영역 (한 페이지에 하나)
`<footer>`	꼬리말, 저작권, 하단 링크
`<article>`, `<section>`	독립된 글 / 주제별 구획 (추가로 알아둘 태그)

div만 쓴 코드 vs 시맨틱 코드

😳 **div만 사용**

```
<div class="header">...</div>
<div class="menu">...</div>
<div class="content">...</div>
<div class="footer">...</div>
```

😜 **시맨틱 사용**

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<footer>...</footer>
```

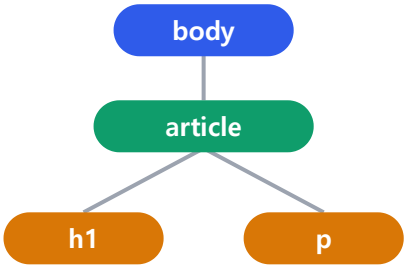
왜 중요한가

구조가 명확할수록 검색엔진·화면낭독기·AI가 내용을 정확히 파악하고 인용하기 쉽습니다. (슬라이드의 "AEO 강의의 '구조화'와 바로 이어집니다" 부분이 바로 이것입니다. AEO는 AI 답변에 잘 인용되도록 콘텐츠를 최적화하는 개념입니다.)

2-6. 중첩과 계층: 요소는 나무처럼 겹쳐 자란다

요소 안에 다른 요소를 넣을 수 있습니다. 이를 **중첩(nesting)**이라고 하고, 중첩 때문에 문서 전체가 **나무(트리) 구조**를 이룹니다.

```
<body>
  <article>
    <h1>제목</h1>
    <p>문단</p>
  </article>
</body>
```



- 바깥 요소 = **부모**, 안쪽 요소 = **자식**. 위 예에서 `article`은 `body`의 자식이면서 `h1`, `p`의 부모입니다.
- `h1`과 `p`는 같은 부모를 둔 **형제** 관계입니다.
- 들여쓰기(indent)는 규칙이 아니라 사람이 읽기 쉽게 하는 습관입니다. 하지만 구조를 눈으로 확인하기 위해 꼭 지키는 것이 좋습니다.

주의 — 닫는 순서가 어긋나면 안 된다

```
❌ <p>안녕 <b>반가워</p></b>      (b가 p 밖으로 걸침)
✅ <p>안녕 <b>반가워</b></p>      (안쪽부터 먼저 닫는다)
```

먼저 연 것을 나중에 닫는 "러시아 인형" 규칙을 기억하세요.

2-7. DOM: 브라우저가 만든 문서 트리

핵심 정의

DOM(Document Object Model)은 브라우저가 HTML을 읽어서 메모리 안에 만들어 놓은 **트리 형태의 구조물**입니다. JavaScript는 이 DOM을 바꿔서 화면을 동적으로 조작합니다.

HTML

파일에 적힌 **설계도(텍스트)**. 정적인 코드. 파일을 바꾸지 않는 한 그대로.

DOM

설계도로 지은 **실제 구조물**. JS가 실시간으로 손댈 수 있는 대상.

쉽게 말하면

건축 도면(HTML)과 실제 지어진 건물(DOM)의 관계입니다. 도면은 종이에 그려진 그림이지만, 건물은 벽을 허물고 가구를 옮길 수 있습니다. 브라우저는 도면을 보고 건물을 짓고, JS는 그 건물을 리모델링합니다. 이때 종이 도면은 바뀌지 않습니다 — 바뀌는 건 건물(DOM)뿐입니다.

직접 해보기 ③ (DOM의 증거)

아무 사이트에서 F12 → Elements 탭에서 글자를 더블클릭해 바꿔 보세요. 화면의 글자가 즉시 바뀝니다. 새로고침하면 원래대로 돌아옵니다. 서버의 HTML 원본이 아니라 **브라우저 안의 DOM**만 고쳤기 때문입니다.

🖍️ PART 2 확인 문제

Q5. 아래 코드에서 (가) 태그, (나) 요소, (다) 속성 에 해당하는 부분을 각각 짚어 쓰세요.

```
<img src="logo.png" alt="회사 로고">
```

속성은 두 개입니다. 각각의 역할도 설명하세요.

Q6. `<head>`와 `<body>`의 차이를 설명하세요.

Q7. 다음 코드는 중첩이 잘못되었습니다. 올바르게 고쳐 쓰세요.

```
<ul><li>사과</ul></li>
```

Q8. 다음 페이지 영역에 가장 알맞은 시맨틱 태그를 연결하세요.

(a) 상단 로고와 사이트명 → _____ (b) 메뉴 링크 모음 → _____ (c) 글의 핵심 본문 → _____ (d) 저작권 문구 → _____

Q9. "HTML 파일"과 "DOM"의 차이를 건축에 빗대어 설명해 보세요.

PART 3

HTML · CSS · JS 역할 분담

웹 페이지는 세 언어의 협업으로 완성됩니다 — 구조·표현·동작이 각자 맡은 일을 합니다.

3-1. 웹의 세 기둥: 구조 · 표현 · 동작

HTML
구조 · 뼈대
무엇이 제목·문단·
이미지인지 정한다

CSS
표현 · 꾸밈
색·크기·여백·배치 등
보이는 스타일을 입힌다

JavaScript
동작 · 움직임
클릭·입력에 반응하는
동작과 상호작용 담당

그림 3-1. 웹의 세 기둥

쉽게 말하면
사람으로 치면 HTML은 **뼈대**, CSS는 **옷과 화장**, JS는 **움직이는 근육**입니다. 뼈대만 있으면 해골이고, 옷을 입혀야 보기 좋고, 근육이 있어야 움직입니다.

	HTML	CSS	JavaScript
역할	구조·내용	스타일·배치	동작·상호작용
질문	"이게 무엇인가?"	"어떻게 보이는가?"	"무엇을 하는가?"
종류	마크업 언어	스타일시트 언어	**프로그래밍 언어**
없으면?	화면에 보일 내용이 없음	못생기지만 동작은 함	정적인 페이지가 됨

3-2. 같은 버튼을 세 언어로 만들어 보기

역할 분담이 가장 잘 이해되는 예시입니다. 하나의 버튼에 세 언어가 각각 무엇을 하는지 보세요.

```
<!-- HTML: 버튼이 "있다"고 선언 -->
<button id="btn">눌러보세요</button>
<p id="msg"></p>

<style>
  /* CSS: 버튼을 "예쁘게" */
  #btn { background: navy; color: white; padding: 10px 20px; border-radius: 8px; }
</style>

<script>
  // JS: 버튼을 누르면 "무엇이 일어나게" 할지
  document.getElementById("btn").addEventListener("click", function () {
    document.getElementById("msg").textContent = "안녕하세요! 클릭하셨네요.";
  });
</script>
```

직접 해보기 ④

위 코드를 메모장에 붙여넣고 `test.html`로 저장한 뒤 크롬으로 열어 보세요. 그다음 ①CSS 부분을 지워 보고(버튼이 못생겨짐), ②JS 부분을 지워 보세요(눌러도 아무 일도 안 일어남). 각 언어의 역할이 눈에 보입니다. (자세한 따라하기는 부록 D에 있습니다.)

3-3. CSS 맛보기: 대상을 고르고 스타일을 준다

```
h1 {
  color: navy;
  font-size: 40px;
}
```

```
h1  {                    ◀ 선택자 : 무엇을 꾸밀지 (모든 h1)
    color  :  navy ;     ◀ 속성 : 값 → 어떻게 보일지 (글자색 : 남색)
}
```

개념	설명
선택자(Selector)	"어떤 요소에 적용할지" 대상을 고름. `h1`(태그 이름), `.notice`(class), `#btn`(id) 등
속성 : 값	"어떻게 보일지" 지정. 색(`color`), 크기(`font-size`), 여백(`margin`) 등. 끝에 `;`
분리 원칙	스타일은 HTML과 **따로** 관리한다. 같은 HTML이라도 CSS만 바꾸면 완전히 다른 디자인이 된다.

CSS를 HTML에 연결하는 세 가지 방법

방법	예시	평가
외부 파일 ★권장	`<link rel="stylesheet" href="style.css">` (head 안)	여러 페이지가 하나의 CSS를 공유. 분리 원칙에 가장 충실
내부 `<style>`	`<style> h1{color:navy;} </style>`	한 페이지에만 쓸 때. 학습·테스트에 편함
인라인 `style` 속성	`<h1 style="color:navy">`	분리 원칙에 어긋나므로 가급적 피함

쉽게 말하면

같은 인형(HTML)에 다른 옷(CSS)을 갈아입히는 것과 같습니다. 인형은 그대로이고 옷만 바꾸면 분위기가 달라지죠. 옷과 인형을 따로 보관하면 관리가 쉽습니다.

3-4. JavaScript 맛보기: 페이지에 생명을 준다

JavaScript(JS)의 핵심은 **사용자의 행동(이벤트)에 반응해 화면(DOM)을 바꾸는 것**입니다. 흔한 활용은 아래와 같습니다.

하는 일	예시
👆 버튼 클릭에 반응	햄버거 메뉴 열기·닫기, 팝업 띄우기
✅ 입력값 검증	이메일 형식이 아니면 "올바른 이메일을 입력하세요" 표시
🔄 서버에서 데이터 요청	댓글, 좋아요 수, 날씨 정보 갱신
✨ 새로고침 없이 일부만 변경	무한 스크롤, 장바구니 숫자 갱신, 자동완성 검색어

이벤트 → 반응 구조

```
이벤트(클릭 등) ──▶ JS가 감지 ──▶ DOM을 수정 ──▶ 화면이 바뀜
```

주의

JavaScript ≠ Java. 이름만 비슷할 뿐 전혀 다른 언어입니다. (인도와 인도네시아 정도의 차이라고 생각해도 됩니다.)

자세히 보기 — "새로고침 없이 바뀐다"는 것의 의미

예전에는 화면의 일부만 바꾸려 해도 서버에서 페이지 전체를 다시 받아야 했습니다. 지금은 JS가 백그라운드에서 필요한 데이터만 서버에 요청(HTTP!)한 뒤 DOM의 일부만 바꿉니다. 그래서 PART 1(HTTP)과 PART 2(DOM)가 JS를 통해 연결됩니다.

🖍️ PART 3 확인 문제

Q10. 다음 각각의 작업은 HTML·CSS·JS 중 어디의 일인가요?
(a) 제목 글자를 파란색으로 _____ (b) "회원가입" 버튼을 만들기 _____ (c) 버튼을 누르면 팝업창 띄우기 _____ (d) 두 칼럼으로 배치하기 _____ (e) 이메일 형식 검사하기 _____

Q11. CSS 코드 `p { color: red; font-size: 18px; }` 에서 선택자, 속성, 값을 구분해 보세요.

Q12. "스타일은 HTML과 따로 관리한다"는 분리 원칙이 왜 좋은지 두 가지 이유를 쓰세요.

Q13. 페이지를 새로고침하지 않고도 화면 일부가 바뀌는 것이 가능한 이유를 DOM과 JS의 관계로 설명해 보세요.

PART 4

브라우저 렌더링과 개발자도구

받아온 코드가 눈에 보이는 화면이 되기까지, 브라우저 안에서 벌어지는 일입니다.

4-1. 코드가 화면이 되는 길: 렌더링 4단계

핵심 정의

렌더링(Rendering)은 브라우저가 서버에서 받은 HTML·CSS·JS 코드를 해석하여 눈에 보이는 화면으로 그려내는 전 과정입니다.

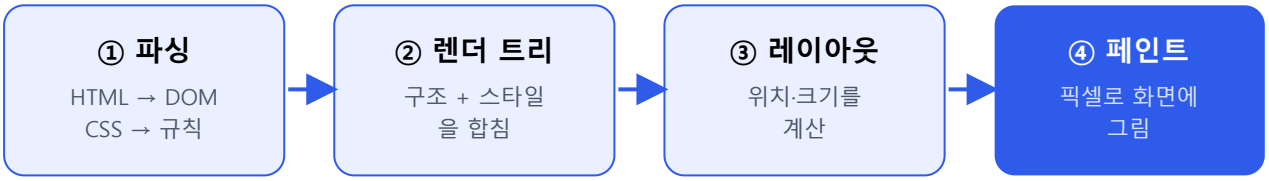


그림 4-1. 렌더링 4단계

단계	하는 일	비유
① 파싱 (Parsing)	받은 HTML 텍스트를 한 줄씩 읽어 **DOM 트리**로 만든다. CSS도 읽어서 "어떤 요소에 어떤 스타일" 규칙(CSSOM)으로 정리한다. *"해석한다"*는 뜻.	설계도(HTML)와 인테리어 지침서(CSS)를 읽고 이해하기
② 렌더 트리 (Render Tree)	DOM(구조)과 CSS 규칙(스타일)을 **합쳐** 화면에 실제로 그릴 요소의 트리를 만든다. (`display:none`처럼 안 보이는 요소는 제외)	도면 + 지침서를 하나의 시공 계획서로 합치기
③ 레이아웃 (Layout)	각 요소의 **정확한 위치와 크기**를 계산한다. 화면 너비가 달라지면 다시 계산.	가구를 방 어디에 얼마만큼 놓을지 치수 재기
④ 페인트 (Paint)	계산된 위치에 색·글자·이미지를 **픽셀로 칠한다**. 이때 비로소 우리 눈에 보인다.	실제로 페인트칠하고 가구 놓기

왜 중요한가 — 성능과 최적화

HTML·CSS가 무겁거나 복잡하면 이 과정이 느려져 **페이지가 늦게 뜹니다**. 그래서 불필요한 코드·큰 이미지를 줄이는 **최적화**가 중요합니다. 웹 성능 개선은 결국 "렌더링 과정을 가볍게 만드는 일"이라고 볼 수 있습니다.

자세히 보기 — 왜 이 순서일까?

HTML만 있으면 "무엇이 있는지"만 알고 "어떻게 생겼는지"는 모릅니다. CSS까지 합쳐야 모양을 알 수 있고(렌더 트리), 모양이 정해져야 자리를 계산할 수 있으며(레이아웃), 자리가 정해져야 칠할 수 있습니다(페인트). 앞 단계의 결과가 다음 단계의 재료가 되는 구조입니다. 또한 JS가 DOM을 바꾸면 이 과정 일부가 다시 실행되는데, 이것이 "화면이 갱신되는" 원리입니다.

4-2. 직접 확인하는 창: 개발자도구(F12)

지금까지 배운 내용은 모두 눈에 안 보이는 것들입니다. 그런데 **개발자도구**를 열면 이 모든 것을 눈으로 확인할 수 있습니다.

여는 방법

크롬/엣지에서 **F12** 키, 또는 화면에서 **우클릭 → 검사**.

탭	무엇을 보나	이 강의와의 연결
Elements	HTML·DOM 구조와 CSS를 **실시간으로 확인·수정**	PART 2 (태그·중첩·DOM), PART 3 (CSS)
Network	요청·응답, **상태 코드**, 로딩 시간을 관찰	PART 1 (HTTP·메서드·상태 코드)
Console	JavaScript 실행, **오류 메시지** 확인	PART 3 (JS)

🔬 실습 가이드: 오늘 배운 것 직접 확인하기

아무 사이트(예: 네이버, 위키백과)에서 F12를 누르고 아래 순서대로 따라해 보세요. 각 실습에서 "✓"가 확인되면 해당 개념을 이해한 것입니다.

실습 1. Elements — DOM 트리 확인하기

1. F12 → **Elements** 탭을 선택한다.
2. `<html>` 옆의 ▶ 화살표를 눌러 펼쳐 보면 `<head>`와 `<body>`가 있다. ✓ (PART 2-2)
3. 왼쪽 위의 "요소 선택(화살표 아이콘)"을 누르고 화면의 제목을 클릭하면, 그 제목에 해당하는 태그로 이동한다. ✓ 태그·요소 (PART 2-3)
4. 글자를 더블클릭해 수정해 본다. 화면이 바뀌지만 새로고침하면 원상복구. ✓ DOM vs HTML (PART 2-7)
5. 오른쪽 Styles 영역에서 `color` 값을 바꿔 본다. ✓ CSS (PART 3-3)

실습 2. Network — 요청과 응답 눈으로 보기

1. **Network** 탭을 열고 **F5**로 새로고침한다. (먼저 탭을 열어 둬야 기록됩니다)
2. 첫 번째 항목이 보통 HTML 문서 요청이다. 클릭해서 **Headers**에서 Request URL, Request Method(`GET`), Status Code(`200`)를 확인한다. ✓ (PART 1-6~1-8)

3. 그 아래 수십 개의 이미지·CSS·JS 요청이 줄줄이 있다. ✓ "페이지 하나를 열 때 요청은 한 번이 아니다" (PART 1-9)
4. 오른쪽 **Time**(또는 Waterfall)에서 어떤 파일이 오래 걸리는지 본다. ✓ 렌더링·최적화 (PART 4-1)
5. 주소 끝에 없는 경로를 붙여 `404`를 직접 만들어 본다. (예: 사이트 주소/asdfasdf) ✓ 상태 코드 (PART 1-8)

실습 3. Console — JS 맛보기

1. **Console** 탭을 열고 아래를 입력한 뒤 Enter.

```
document.title                          // 이 페이지의 <title> 값이 출력됨
document.body.style.background = "pink"   // 페이지 배경이 분홍색으로 바뀜
```

✓ JS가 DOM을 조작해 화면을 즉시 바꾼다는 것을 체감할 수 있습니다. 새로고침하면 사라집니다.

> **주의**
> 콘솔에 모르는 사람이 알려준 코드를 붙여넣지 마세요. 계정 정보를 훔치는 사기에 쓰이는 방법이기도 합니다. 위 예시처럼 내용을 이해할 수 있는 코드만 입력하세요.

🖍️ PART 4 확인 문제

Q14. 렌더링 4단계를 순서대로 쓰고, 각 단계를 한 줄로 설명하세요.

Q15. 이미지가 매우 큰 페이지가 느리게 뜬다면, 개발자도구의 어느 탭에서 원인을 찾아볼 수 있을까요?

Q16. Elements 탭에서 글자를 고쳤는데 새로고침하니 원래대로 돌아왔습니다. 왜 그럴까요?

부록 A · 핵심 용어 사전

용어	원래 말	쉬운 설명
인터넷	Internet	전 세계 컴퓨터를 연결한 통신망(기반)
웹	World Wide Web	인터넷 위에서 문서·페이지를 주고받는 서비스
클라이언트	Client	요청하는 쪽(대표: 브라우저)
서버	Server	요청에 응답하는 쪽. 켜 둔 컴퓨터
URL	Uniform Resource Locator	자원의 주소
도메인	Domain	사람이 읽는 서버 이름 (example.com)
IP 주소	Internet Protocol Address	컴퓨터가 쓰는 숫자 주소 (93.184.216.34)
DNS	Domain Name System	도메인 → IP로 바꿔주는 전화번호부
HTTP	HyperText Transfer Protocol	브라우저와 서버의 통신 규약
HTTPS	HTTP Secure	암호화된 HTTP
메서드	Method	요청의 종류 (GET·POST·PUT·DELETE)
상태 코드	Status Code	응답 결과를 알리는 3자리 숫자
HTML	HyperText Markup Language	페이지 구조를 표시하는 마크업 언어
태그 / 요소 / 속성	Tag / Element / Attribute	표시 / 표시가 감싼 한 덩어리 / 요소의 추가 정보
시맨틱	Semantic	의미를 담은 태그 (header, nav, main, footer)
CSS	Cascading Style Sheets	모양을 꾸미는 스타일 언어
JavaScript	—	동작을 담당하는 프로그래밍 언어
DOM	Document Object Model	브라우저가 만든 문서 트리 구조물
파싱	Parsing	코드를 읽어 의미 있는 구조로 해석하는 것
렌더링	Rendering	코드를 화면으로 그려내는 과정
캐시	Cache	한 번 받은 것을 저장해 두고 재사용하는 것

부록 B · 흔한 오해 바로잡기

✕ 오해	✓ 사실	이유
"HTML은 프로그래밍 언어다"	구조를 표시하는 **마크업 언어**	조건·반복·계산 같은 로직이 없고, 내용에 의미를 붙이는 용도
"인터넷과 웹은 같다"	웹은 인터넷 위의 **서비스 하나**	이메일·메신저·게임도 인터넷을 쓰지만 웹이 아니다
"꼭 새로고침해야 바뀐다"	JS가 **일부만 갱신**할 수 있다	JS가 DOM을 직접 바꾸기 때문
"서버는 특별한 기계다"	요청에 응답하도록 **켜 둔 컴퓨터**일 뿐	서버는 '역할'이다. 내 PC에서도 서버를 돌릴 수 있다
"h1~h6은 글자 크기 조절 태그"	제목의 **중요도 계층**	크기는 CSS로 정한다
"자물쇠가 있으면 완전히 안전한 사이트"	통신이 **암호화**되었다는 뜻일 뿐	사이트 자체가 신뢰할 만한지는 별개 문제 (피싱 사이트도 HTTPS를 쓸 수 있음)
"JavaScript는 Java의 일부다"	전혀 **다른 언어**	이름만 비슷

부록 C · 종합 연습문제

전체를 복습하는 문제입니다. 책을 덮고 풀어본 뒤 부록 E의 해설과 비교하세요.

A. 빈칸 채우기

1. 웹 통신에서 요청하는 쪽을 _____, 응답하는 쪽을 _____라고 한다.

2. 도메인 이름을 IP 주소로 바꿔주는 시스템은 _____이다.

3. HTTP에 암호화를 더한 것을 _____라고 한다.

4. 상태 코드가 _____로 시작하면 클라이언트 잘못, _____로 시작하면 서버 잘못이다.

5. HTML 문서에서 화면 밖 정보를 담는 부분은 _____, 화면에 보이는 내용을 담는 부분은 _____이다.

B. O / X

6. 웹과 인터넷은 같은 말이다. ( )

7. 브라우저는 서버가 보낸 HTML을 읽어 DOM이라는 트리 구조를 만든다. ( )

8. `<div>`는 의미를 담은 시맨틱 태그이다. ( )

9. CSS는 페이지의 구조를 정하고, HTML은 페이지를 꾸민다. ( )

10. 렌더링의 마지막 단계는 페인트이다. ( )

C. 서술형

11. 주소창에 `https://www.example.com` 을 입력하고 Enter를 눌렀을 때 일어나는 일을 5단계로 설명하세요.

12. 시맨틱 태그를 쓰면 좋은 점을 설명하세요.

13. 웹 개발 비전공자인 친구에게 "HTML, CSS, JS가 뭐가 다른지"를 사람의 몸에 빗대어 설명해 보세요.

부록 D · 미니 프로젝트: 내 첫 웹 페이지

이론을 손으로 확인하는 시간입니다. 설치할 것은 **없습니다**. 메모장과 크롬만 있으면 됩니다.

Step 1. 파일 만들기

1. 바탕화면에 `myweb` 폴더를 만든다.
2. 메모장을 열고 아래 코드를 그대로 붙여넣는다.
3. **다른 이름으로 저장** → 파일 이름 `index.html`, 인코딩 **UTF-8**, 파일 형식 **모든 파일**로 저장한다.
4. 저장된 파일을 더블클릭하면 크롬에서 열린다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>나의 첫 웹 페이지</title>
  <style>
    body   { font-family: sans-serif; margin: 0; }
    header { background: navy; color: white; padding: 20px; }
    nav a  { margin-right: 12px; }
    main   { padding: 20px; }
    footer { background: #eee; padding: 10px 20px; font-size: 14px; }
    #btn   { background: navy; color: white; border: none; padding: 10px 20px; border-
radius: 8px; }
  </style>
</head>
<body>
  <header>
    <h1>나의 첫 웹 페이지</h1>
  </header>

  <nav>
    <a href="#intro">소개</a>
    <a href="https://example.com">외부 링크</a>
  </nav>

  <main>
    <h2 id="intro">자기소개</h2>
    <p>안녕하세요! 웹 기본지식을 배우고 있어요.</p>
    <ul>
      <li>HTML은 뼈대</li>
      <li>CSS는 꾸밈</li>
      <li>JS는 동작</li>
    </ul>
    <button id="btn">눌러보세요</button>
    <p id="msg"></p>
  </main>

  <footer>&copy; 2026 나의 웹 페이지</footer>

  <script>
    document.getElementById("btn").addEventListener("click", function () {
      document.getElementById("msg").textContent = "클릭했습니다! JS가 DOM을 바꿨어요.";
    });
  </script>
</body>
</html>
```

Step 2. 따라 해보기 미션

☐ **미션 1** 브라우저 탭의 제목이 "나의 첫 웹 페이지"로 나오는지 확인한다. (`<title>`)

☐ **미션 2** `<h1>` 의 텍스트를 내 이름으로 바꾸고 저장 후 **F5**로 새로고침한다.

☐ **미션 3** `header` 의 배경색 `navy` 를 `tomato` 로 바꿔 본다. (CSS)

☐ **미션 4** 목록에 `<li>` 하나를 더 추가한다. (예: "DOM은 살아있는 구조물")

☐ **미션 5** `<img src="https://picsum.photos/200" alt="랜덤 이미지">` 한 줄을 `main` 안에 넣는다. 인터넷에 연결되어 있어야 보입니다.

☐ **미션 6** F12 → Elements 탭에서 `header` , `nav` , `main` , `footer` 가 나란히 형제인 것을 확인한다. (중첩·계층)

☐ **미션 7** Network 탭을 열고 새로고침한 뒤, `index.html` 요청과 (이미지를 넣었다면) 이미지 요청이 따로 있는지 확인한다.

☐ **미션 8** "눌러보세요" 버튼을 누른 뒤 Elements에서 `<p id="msg">` 안에 글자가 생겼는지 확인한다. (DOM이 바뀜) 그리고 소스 보기(**Ctrl+U**)에서는 그 글자가 없다는 점을 비교한다. (HTML 원본 ≠ DOM)

☐ **미션 9** `</footer>` 닫는 태그 하나를 일부러 지워서 어떻게 되는지 관찰한다. (HTML은 오류에 관대하지만 구조가 틀어질 수 있음)

> **여기까지 해냈다면**
>
> HTML(구조) → CSS(스타일) → JS(동작) → DOM → 개발자도구까지 한 바퀴를 직접 돌아본 것입니다. 이 워크북의 핵심 개념이 모두 손으로 확인되었습니다.

> **참고**
>
> 이 파일은 내 컴퓨터에서 직접 연 것이라 `file:///...` 주소로 보이며, 서버와 HTTP로 통신한 것이 아닙니다. 실제 웹 서버에 올려야 URL·DNS·HTTP 과정이 일어납니다. 지금은 "내 브라우저가 HTML을 렌더링하는 과정"만 체험한 것입니다.

셀프 체크리스트

☐ 인터넷과 웹의 차이를 설명할 수 있다

☐ 클라이언트와 서버의 요청·응답 관계를 설명할 수 있다

☐ URL의 네 부분을 구분할 수 있다

☐ DNS가 하는 일을 설명할 수 있다

☐ HTTP와 HTTPS의 차이를 설명할 수 있다

☐ GET/POST/PUT/DELETE의 용도를 말할 수 있다

☐ 2xx, 3xx, 4xx, 5xx 상태 코드의 의미를 구분할 수 있다

☐ HTML 기본 골격(DOCTYPE, html, head, body)을 직접 쓸 수 있다

☐ 태그·요소·속성을 구분할 수 있다

☐ 시맨틱 태그가 필요한 이유를 설명할 수 있다

- ☐ HTML 파일과 DOM의 차이를 설명할 수 있다
- ☐ HTML·CSS·JS의 역할 차이를 설명할 수 있다
- ☐ 렌더링 4단계를 순서대로 말할 수 있다
- ☐ F12의 Elements/Network/Console을 열어 볼 수 있다

부록 E · 정답 및 해설

서술형은 "이런 내용이 들어가면 정답"이라는 예시입니다. 표현은 달라도 핵심이 맞으면 됩니다.

PART 1

1. **Q1.** 인터넷은 전 세계 컴퓨터를 잇는 통신망 자체이고, 웹은 그 통신망 위에서 브라우저로 페이지를 주고받는 서비스 중 하나이다.
2. **Q2.** 프로토콜 `https://` / 도메인 `news.example.org` / 경로 `/sports/soccer` / 쿼리 `?page=2`
3. **Q3.** (a) GET (b) POST (c) PUT (d) DELETE
4. **Q4.** 404는 "없는 페이지"로 **클라이언트(요청) 쪽** 문제(주소 오타·삭제된 페이지 등). 500은 **서버 내부 오류**로 서버 쪽 문제일 가능성이 크다. (단, 404가 사이트 쪽 링크 관리 실수일 수도 있어 "가능성이 크다" 수준으로 판단한다.)

PART 2

5. **Q5.** 태그: `<img ...>` 자체(이 요소는 닫는 태그가 없는 빈 요소). 요소: 그 전체 한 덩어리(`<img src="logo.png" alt="회사 로고">`). 속성: `src="logo.png"`(이미지 파일 위치), `alt="회사 로고"` (이미지 대체 설명 — 화면낭독기·로딩 실패 시 사용).
6. **Q6.** head는 제목·인코딩·CSS 연결 등 **화면에 직접 보이지 않는 문서 정보**, body는 **화면에 보이는 실제 내용**.
7. **Q7.** `<ul><li>사과</li></ul>` (안쪽 li를 먼저 닫고 바깥 ul을 닫는다)
8. **Q8.** (a) `header` (b) `nav` (c) `main` (또는 article) (d) `footer`
9. **Q9.** HTML 파일은 건축 **설계도**(글로 적힌 정적 문서)이고 DOM은 설계도대로 지어진 **건물**(브라우저 안의 살아있는 구조)이다. JS는 건물(DOM)을 고쳐 화면을 바꾸며 설계도 파일은 그대로다.

PART 3

10. **Q10.** (a) CSS (b) HTML (c) JS (d) CSS (e) JS
11. **Q11.** 선택자 `p` / 속성 `color`, 값 `red` / 속성 `font-size`, 값 `18px`
12. **Q12.** 예: ① 디자인만 바꾸고 싶을 때 HTML을 건드리지 않아도 된다 ② 하나의 CSS 파일을 여러 페이지가 공유해 관리가 쉽고 일관성이 유지된다 ③ 역할이 분리되어 협업·유지보수가 편하다.
13. **Q13.** 브라우저는 HTML을 DOM으로 만들어 두고, JS는 서버에서 필요한 데이터만 받아 DOM의 **일부만** 수정한다. 그러면 브라우저가 바뀐 부분만 다시 그리므로 페이지 전체를 다시 받을 필요가 없다.

PART 4

14. **Q14.** ① 파싱(HTML→DOM, CSS→규칙으로 해석) → ② 렌더 트리(구조+스타일 합침) → ③ 레이아웃(위치·크기 계산) → ④ 페인트(픽셀로 화면에 그림)
15. **Q15. Network** 탭. 각 파일의 크기와 로딩 시간을 보여주므로 가장 오래 걸리거나 용량이 큰 이미지 파일을 찾을 수 있다.

16. **Q16.** Elements 탭의 수정은 브라우저 안의 **DOM**만 바꾼 것이고 서버의 HTML 원본은 그대로이기 때문이다. 새로고침하면 서버에서 원본 HTML을 다시 받아 DOM을 새로 만든다.

부록 C 종합 연습문제

1. **A-1.** 클라이언트 / 서버
2. **A-2.** DNS
3. **A-3.** HTTPS
4. **A-4.** 4 / 5
5. **A-5.** head / body
6. **B-6.** X — 웹은 인터넷 위의 서비스 하나이다.
7. **B-7.** O
8. **B-8.** X — div는 의미가 없는 범용 상자이다. 시맨틱 태그는 header·nav·main·footer 등.
9. **B-9.** X — 반대이다. HTML이 구조, CSS가 꾸밈.
10. **B-10.** O — 파싱→렌더 트리→레이아웃→페인트
11. **C-11.** ① URL 입력 후 Enter ② DNS로 도메인→IP 변환 ③ 그 서버에 HTTP(S) 요청 전송 ④ 서버가 상태 코드와 함께 HTML 등을 응답 ⑤ 브라우저가 렌더링해 화면에 그림.
12. **C-12.** 이름만 보고 영역의 역할을 알 수 있어 코드 가독성이 좋아지고, 검색엔진·화면낭독기·AI가 구조를 정확히 이해해 검색 노출과 접근성이 좋아진다.
13. **C-13.** HTML은 뼈대(몸의 구조), CSS는 옷과 화장(겉모습), JS는 근육(움직임). 뼈대 없이는 몸이 성립하지 않고, 옷이 없어도 몸은 있지만 보기에 투박하며, 근육이 없으면 움직일 수 없다.

다음 단계 추천

이 워크북의 개념이 이해되었다면, ① HTML 폼(`<form>` , `<input>`) ② CSS 박스 모델과 Flexbox ③ JavaScript 기초(변수·함수·이벤트) ④ 쿠키·세션(로그인 상태 유지) 순서로 학습하면 자연스럽게 이어집니다.

AGM 1기 · 웹 기본지식 워크북 — 수고하셨습니다.